



# ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

## DATA ANALYSIS WITH PANDAS - SOLUTION

**Professor:** *Gabriel E. Cabrera*  
**Teaching Assistant:** *Alex Den Braber*



The Pandas library is designed for tabular data analysis. It not only provides useful classes and functions, it also allows you to apply functions from other libraries such as NumPy.

There are two fundamental structures in Pandas:

Table 1: Fundamental Structures

| Object type      | Description                                         | Used for                          |
|------------------|-----------------------------------------------------|-----------------------------------|
| <b>DataFrame</b> | 2-dimensional object with an index ( <b>index</b> ) | Tabular data organized in columns |
| <b>Series</b>    | 1-dimensional object with an index ( <b>index</b> ) | A single (time) series of data    |

To import Pandas:

```
# pandas is imported
import pandas as pd
```

## 1 Series

A Series is a one-dimensional *array* that contains a sequence of values (as in NumPy) and a *label* called the index (**index**). To create a Series:

```
# a Series is created
obj = pd.Series([4, 7, -5, 3],          # the data is defined
                 index = ['a', 'b', 'c', 'd']) # the index is specified
```

```
# obj is checked
obj
```

```
## a    4
## b    7
## c   -5
## d    3
## dtype: int64
```

## 2 DataFrames

A DataFrame is a 2-dimensional structure with labeled data (*labels*), an index on the rows as well as on the columns. A column can potentially contain different data types. To create a **DataFrame** object:

```
# a DataFrame is created
df = pd.DataFrame([10, 20, 30, 40],          # the data is defined
                  columns = ['numbers'],      # column name
                  index = ['a', 'b', 'c', 'd']) # the index is specified

# df is checked
df
```

```
##      numbers
## a         10
## b         20
## c         30
## d         40
```

It is important to note that:

- The data is organized in columns (they can have custom names)
- There is an index that can take different formats (e.g. numbers, *strings*, etc.).

To access the index:

```
# extracts column
df.columns
```

```
## Index(['numbers'], dtype='object')
```

A Series can be passed to a DataFrame:

```
# series to df
obj_to_df = obj.to_frame()

# the column name is assigned to it
obj_to_df.columns = ['numbers']

# checks obj_to_df
obj_to_df
```

```
##      numbers
## a          4
## b          7
## c         -5
## d          3
```

In summary, the DataFrame structure:

The diagram illustrates the structure of a DataFrame. It shows a table with 9 columns: Name, Team, Number, Position, Age, Height, Weight, College, and Salary. The rows are indexed from 0 to 6. Annotations include: 'Column names' pointing to the column headers, 'Columns axis=1' pointing to the column headers, 'Index label' pointing to the row index, 'Index axis=0' pointing to the row index, 'Missing value' pointing to a 'NaN' value in the 'Number' column, and 'Data' pointing to the data values in the 'Weight' column.

|   | Name            | Team           | Number | Position | Age  | Height | Weight | College           | Salary    |
|---|-----------------|----------------|--------|----------|------|--------|--------|-------------------|-----------|
| 0 | Avery Bradley   | Boston Celtics | 0.0    | PG       | 25.0 | 6-2    | 180.0  | Texas             | 7730337.0 |
| 1 | John Holland    | Boston Celtics | 30.0   | SG       | 27.0 | 6-5    | 205.0  | Boston University | NaN       |
| 2 | Jonas Jerebko   | Boston Celtics | 8.0    | PF       | 29.0 | 6-10   | 231.0  | NaN               | 5000000.0 |
| 3 | Jordan Mickey   | Boston Celtics | NaN    | PF       | 21.0 | 6-8    | 235.0  | LSU               | 1170960.0 |
| 4 | Terry Rozier    | Boston Celtics | 12.0   | PG       | 22.0 | 6-2    | 190.0  | Louisville        | 1824360.0 |
| 5 | Jared Sullinger | Boston Celtics | 7.0    | C        | NaN  | 6-9    | 260.0  | Ohio State        | 2569260.0 |
| 6 | Evan Turner     | Boston Celtics | 11.0   | SG       | 27.0 | 6-7    | 220.0  | Ohio State        | 3425510.0 |

To see the documentation of the Pandas library go to: <https://pandas.pydata.org/docs/>

### 3 Application

1. The files `gapminder.xlsx` and `gapminder.dta` contain an extract from the Gapminder project on life expectancy (`lifeExp`), GDP per capita (`gdpPercap`) and population (`pop`), by country (`continent`). Using the Pandas library, load both datasets into your workspace. Name one of the two `DataFrame` objects `gapminder`.

```
# imports pandas
import pandas as pd

# "reads" xlsx
gapminder = pd.read_excel('gapminder.xlsx')

# "reads" dta, stata format
pd.read_stata('gapminder.dta')
```

2. Show the first and last 10 observations of `gapminder`.

```
# shows the first 10 observations
gapminder.head()

# shows the last 10 observations
gapminder.tail()
```

3. Generate a new `DataFrame` that contains only the countries of the American continent (`americas`) in the year 2007.

```
# extracts columns
gapminder.columns

# extracts the column index of the continent column
gapminder.columns.get_loc('continent')

# extracts index
gapminder.index

# unique values of the continent variable
gapminder.continent.unique()

'''
Selecting variables (columns) from a DataFrame
'''

# way 1:
# selects the continent variable as a series
gapminder.continent

# way 2:
# selects the continent variable as a series
gapminder['continent']

# way 3:
# selects the continent variable as a series using loc (rows can be selected)
gapminder.loc[:, 'continent']
# selects the continent variable as a series using iloc (rows can be selected)
gapminder.iloc[:, 1]

# way 4:
# selects the continent and lifeExp variables using loc (rows can be selected)
gapminder.loc[:, ['continent', 'lifeExp']]
```

```

# selects the continent and lifeExp variables using iloc (rows can be selected)
gapminder.iloc[:,[1,3]]

# way 5:
# selects the variables from continent up to pop (inclusive)
# using loc (rows can be selected)
gapminder.loc[:, 'continent': 'pop']
# selects the variables from continent up to pop (4)
# using iloc (rows can be selected)
gapminder.iloc[:, 1:5]

# way 6:
# selects the continent and lifeExp variables (rows cannot be selected)
gapminder[['continent', 'lifeExp']]

'''
Selecting variables (columns) from a DataFrame
'''

# filters continent == "Americas"
gapminder[gapminder['continent'] == "Americas"]

# filters continent == "Americas" or filters continent == "Asia"
gapminder[(gapminder['continent'] == "Americas") | (gapminder['continent'] == "Asia")]

# filters (continent == "Americas" or filters continent == "Asia") and year == 2007
part_a = ((gapminder['continent'] == "Americas") | (gapminder['continent'] == "Asia"))
part_b = (gapminder['year'] == 2007)
gapminder[part_a & part_b]

# filters continent == "Americas" or filters year == 2007
gapminder1 = gapminder[(gapminder['continent'] == "Americas") & (gapminder['year'] == 2007)]

```

4. Based on the DataFrame generated in (3), show:

- The country with the highest GDP per capita in the year 2007
- The country with the lowest GDP per capita in the year 2007

What do you notice about the index?

```

# sorts the values in descending order and shows the first observation
gapminder1.sort_values(['gdpPercap'], ascending=False).head(1)

# sorts the values in ascending order and shows the last observation
gapminder1.sort_values(['gdpPercap'], ascending=False).tail(1)

```

5. Reset the index of the DataFrame generated in (3). Then drop the variables country and continent.

```

# resets the index and "drops" it
gapminder1 = gapminder1.reset_index(drop=True)

# deletes/removes the continent and year variable (column) (axis=1)
gapminder1 = gapminder1.drop(['continent', 'year'], axis = 1)

# deletes/removes the row with index 0 and 1 (axis=0)
gapminder1.drop([0,1], axis=0)

# adds country as the index, does it in-place
gapminder1.set_index('country', inplace=True)

```

```
# deletes/removes the row whose index equals Ecuador
gapminder1.drop(['Ecuador'], axis=0)
```

6. Using a *list comprehension*, rename the columns with their original name in lowercase.

```
# re-assigns to the column its original name but in lowercase
gapminder1.columns = [i.lower() for i in list(gapminder1.columns)]

# renames the column country to nation
gapminder1.rename(columns={'country': 'nation'})
```

7. Generate a variable that contains life expectancy (lifeexp) in months. Do it by defining a function and by using an anonymous function.

```
# generates the variable lifeexp * 12 and assigns it to the DataFrame gapminder1
gapminder1['lifeexp_month1'] = gapminder1['lifeexp'] * 12

# generates the variable lifeexp * 12 and assigns it to the DataFrame gapminder1
# using an anonymous function
gapminder1['lifeexp_month2'] = gapminder1['lifeexp'].apply(lambda x: x * 12)

# generates the variable lifeexp * 12 and assigns it to the DataFrame gapminder1
# by first defining a function
def monthly(x):
    res = x * 12
    return(res)

# if a column is not selected it applies it to the whole DataFrame
gapminder1['lifeexp_month3'] = gapminder1['lifeexp'].apply(monthly)
```

8. Using the original dataset:

- a. Work only with the European countries.

```
df_europe = gapminder[(gapminder['continent'] == "Europe")].copy()
df_europe.reset_index(drop=True, inplace=True)
```

- b. Generate the growth of GDP per capita by country.

```
# groups by country and applies the growth in an anonymous function
df_europe['gdpgrowth'] = df_europe.groupby('country')['gdpPercap'].apply(lambda x: (x / x.shift(1) - 1))
```

- c. Drop the NAs.

```
# selects a subset (gdpgrowth) and drops them in-place
df_europe.dropna(subset=['gdpgrowth'], inplace=True)
# drops all NAs (row and column)
df_europe.dropna()
```

- d. Build a brief descriptive statistics summary by country.

```
# grouped mean by country
df_europe.groupby('country').mean()
# describe by country, grouped
desc_stat = df_europe.groupby(['country']).describe()
```

- e. Save the descriptive statistics in a file with the .xlsx (excel) extension.

```
# exports the descriptive statistics to excel
desc_stat.to_excel("desc_stat_europe.xlsx")
```